IronLink

GPUDirect-enabled Confidential Virtual Machines via Trusted SmartNICs

Simon Weckler
Advisor: Teofil Bodea
Chair of Computer Systems
https://dse.in.tum.de/



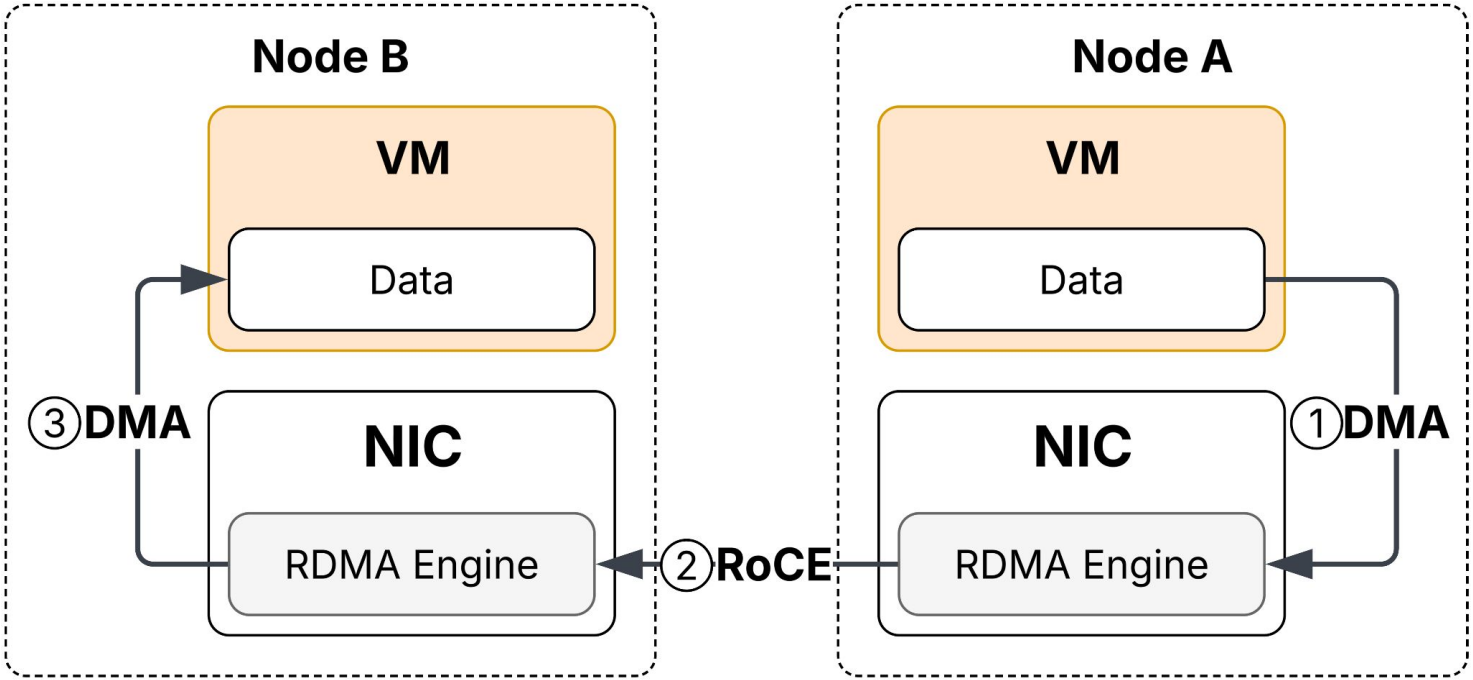
18.05.2026 – 18.09.2026

Outline

- Motivation
  - (GPUDirect) RDMA
  - Confidential Computing
  - Secure Fallback
- Design
- Evaluation

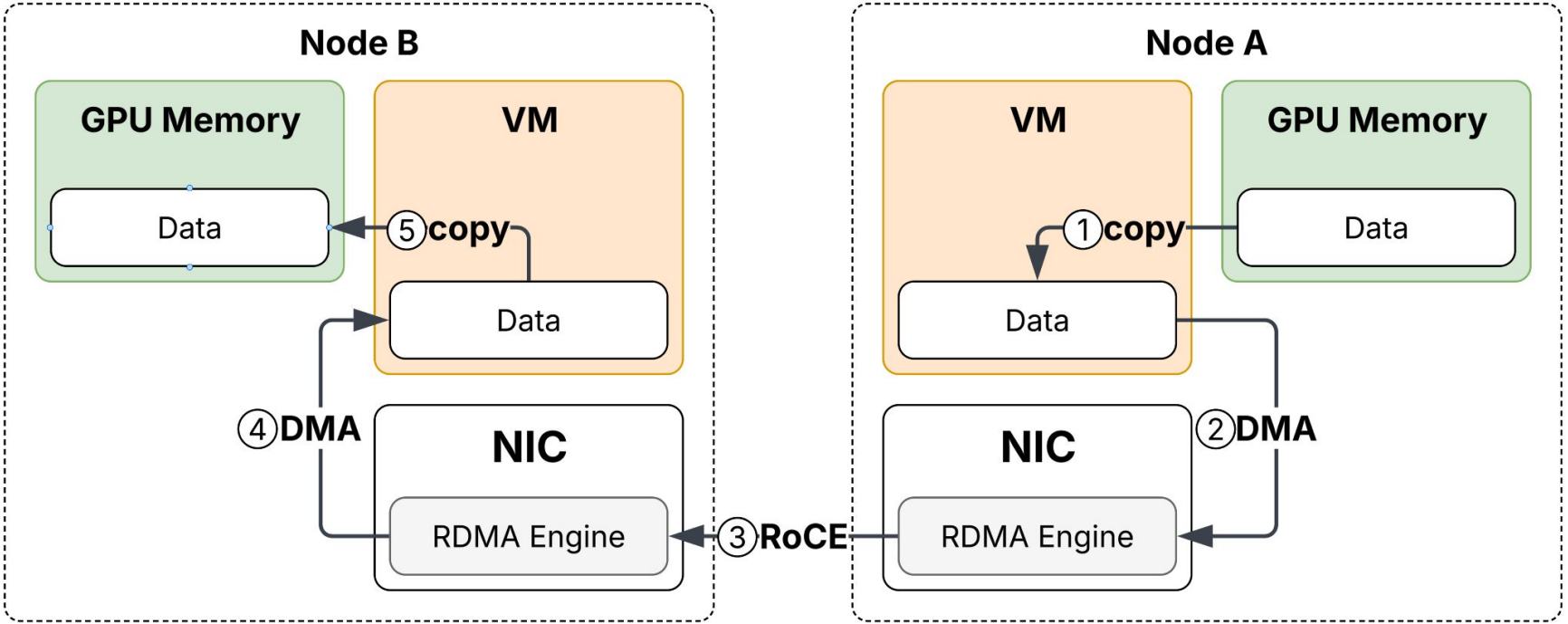
RDMA

Remote Direct Memory Access (RDMA) allows network cards to transfer data directly between memory regions on different machines.



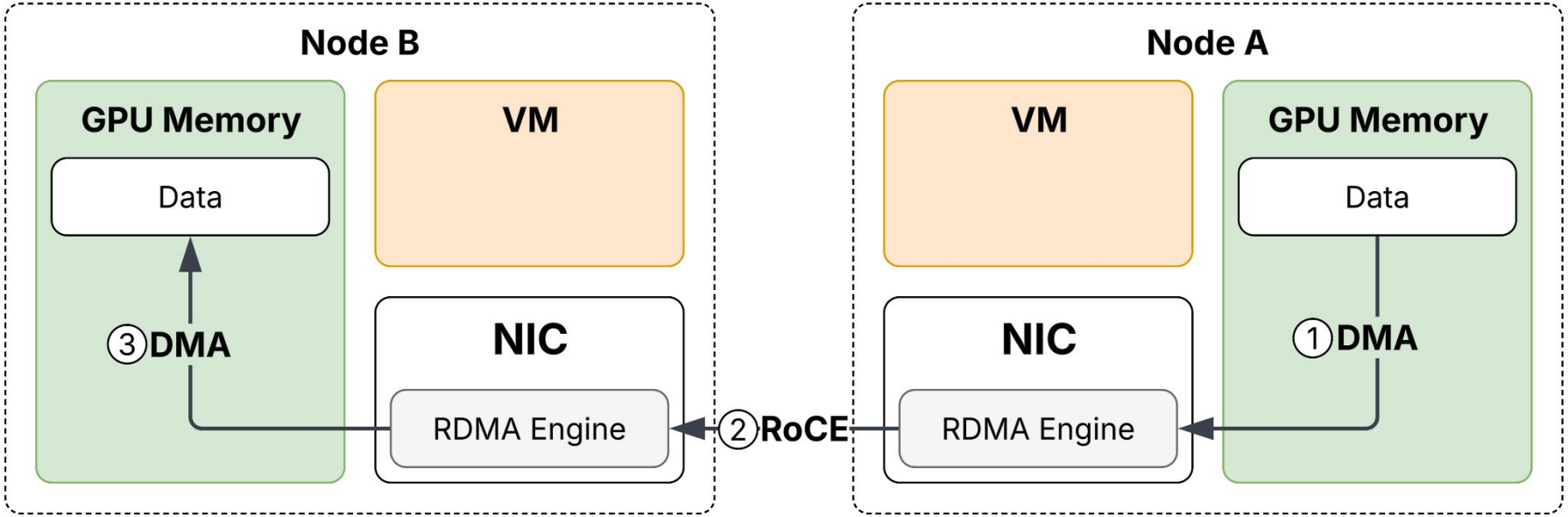
GPU transfers through host memory

RDMA traditionally operate on system memory. GPU data lives in separate device memory (VRAM).



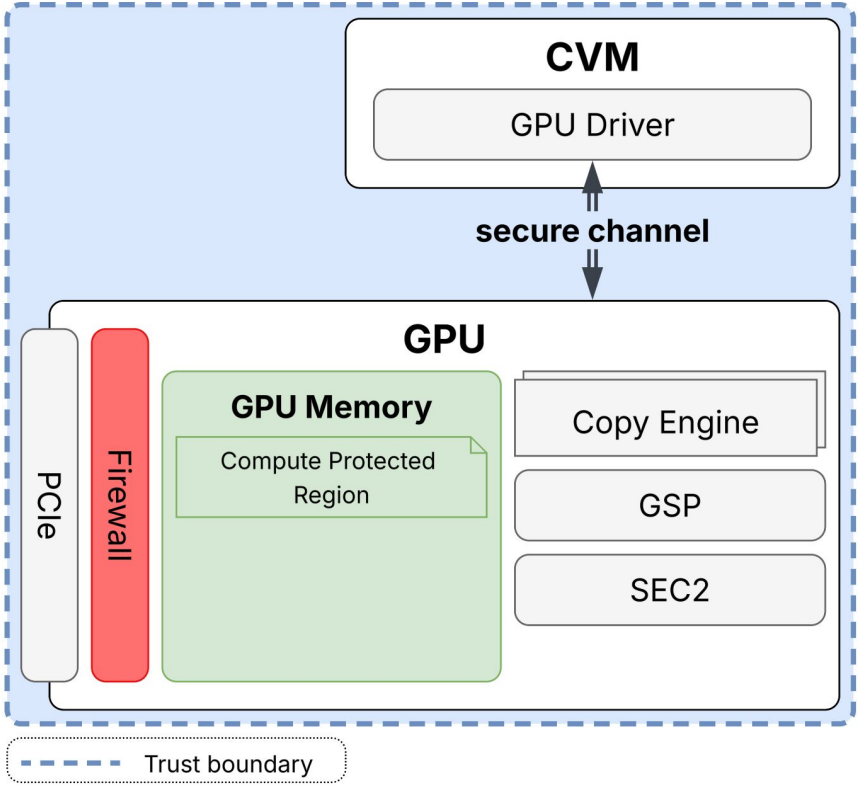
GPUDirect RDMA

GPUDirect RDMA enables data to be transferred directly between GPU memory without staging through host memory.



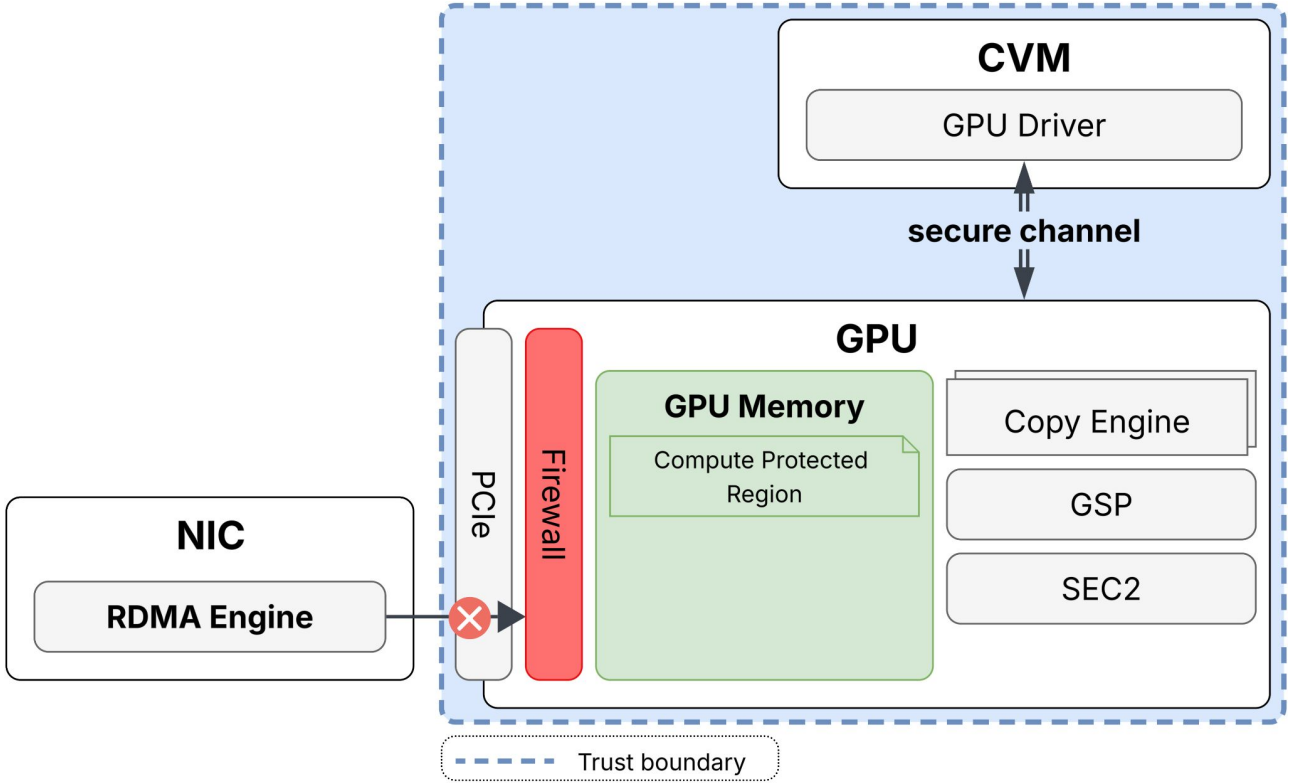
Confidential Computing

Confidential computing (CC) protects application data from the cloud host.



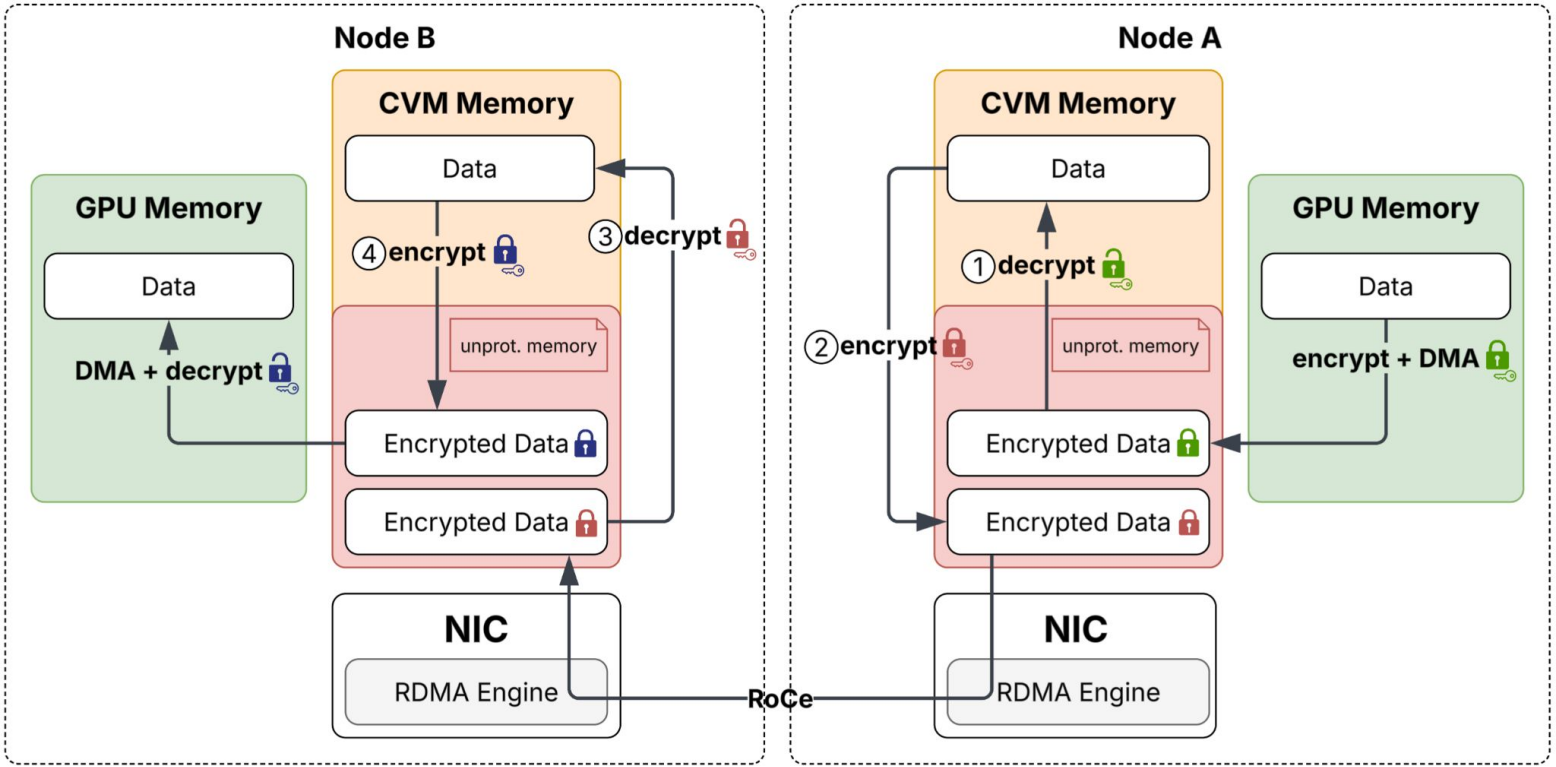
GPUDirect RDMA in CC mode

In CC mode, the GPU firewall blocks ordinary peer to peer access to protected memory.



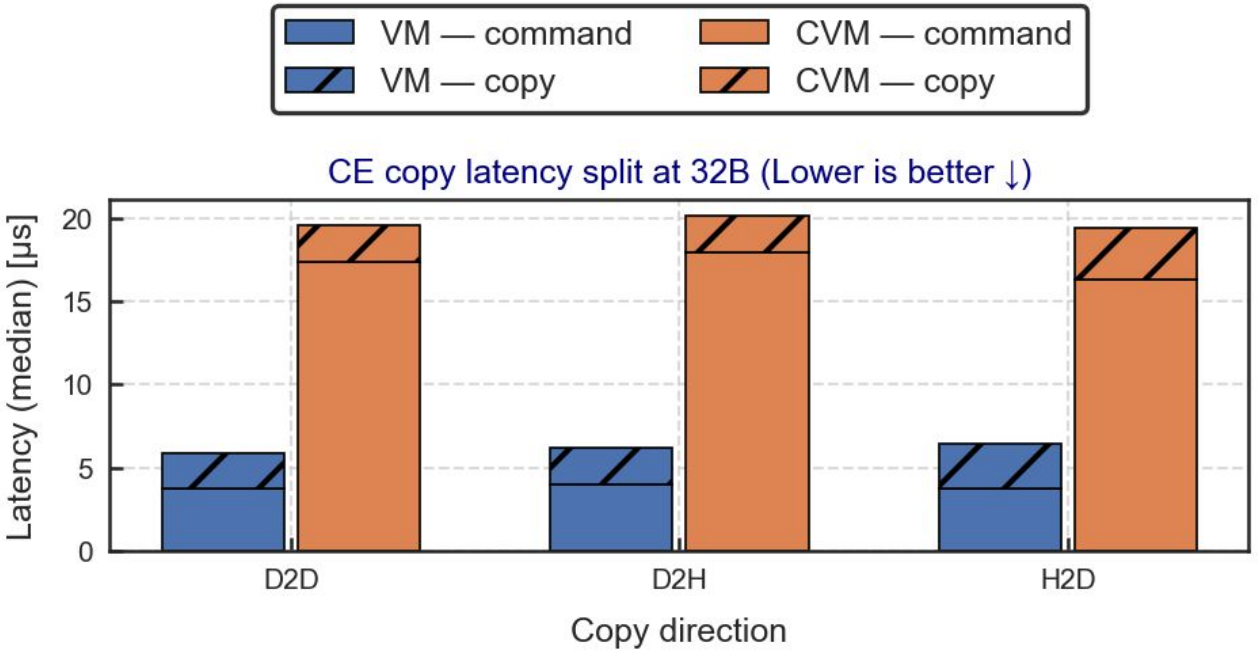
Secure Fallback

Staging through host memory adds 4 CPU cryptographic passes per transfer…



Secure Fallback

…and introduces substantial fixed copy latency (at 32B command overhead accounts for 89% of total D2D copy latency).



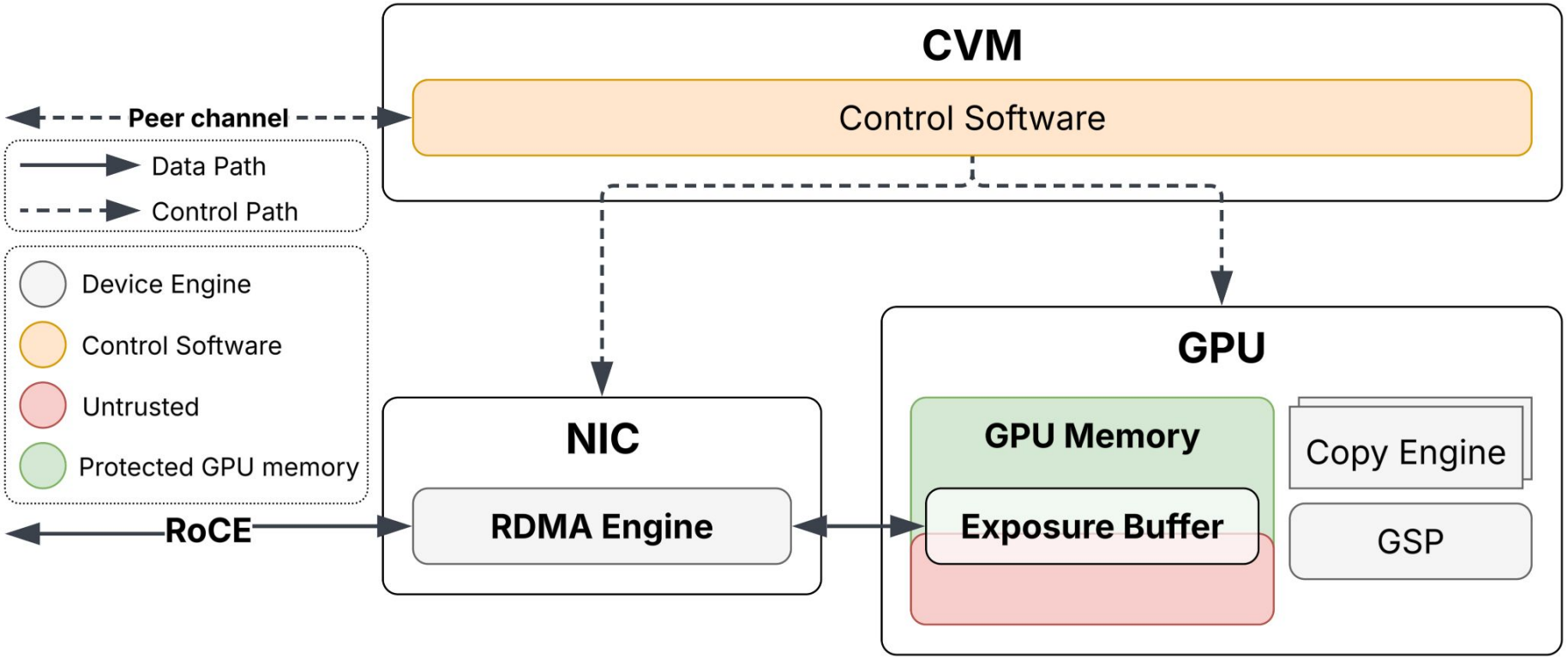
Outline

- ~~Motivation~~
- Design
  - Overview
  - Design 1: In GPU Bounce
  - Design 2: Trusted NIC
- Evaluation

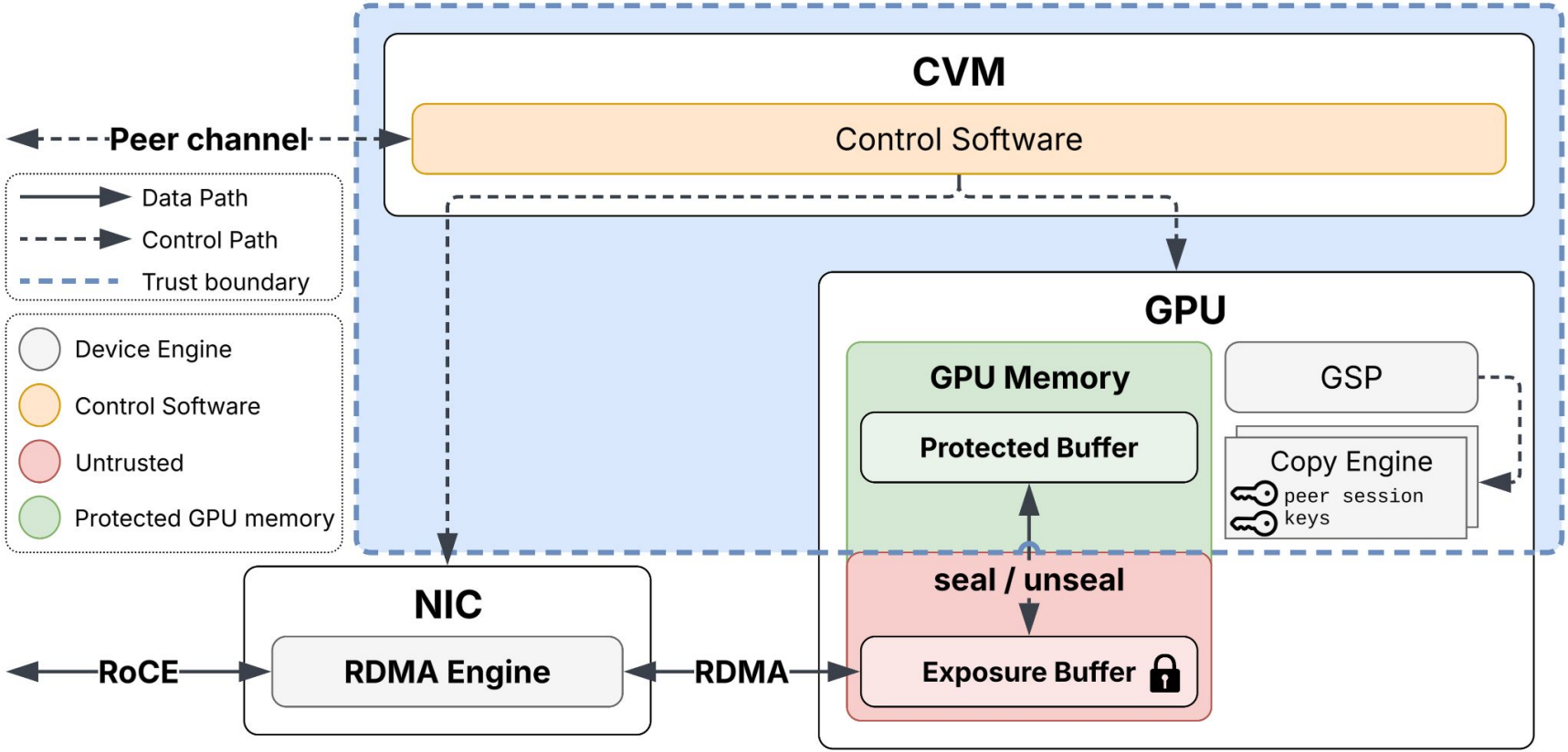
Design Goals

1. Secure end-to-end transfers between attested endpoints
2. Efficient payload protection using GPU or NIC hardware
3. GPUDirect memory transfers without staging payload through host memory
4. Low overhead transfers

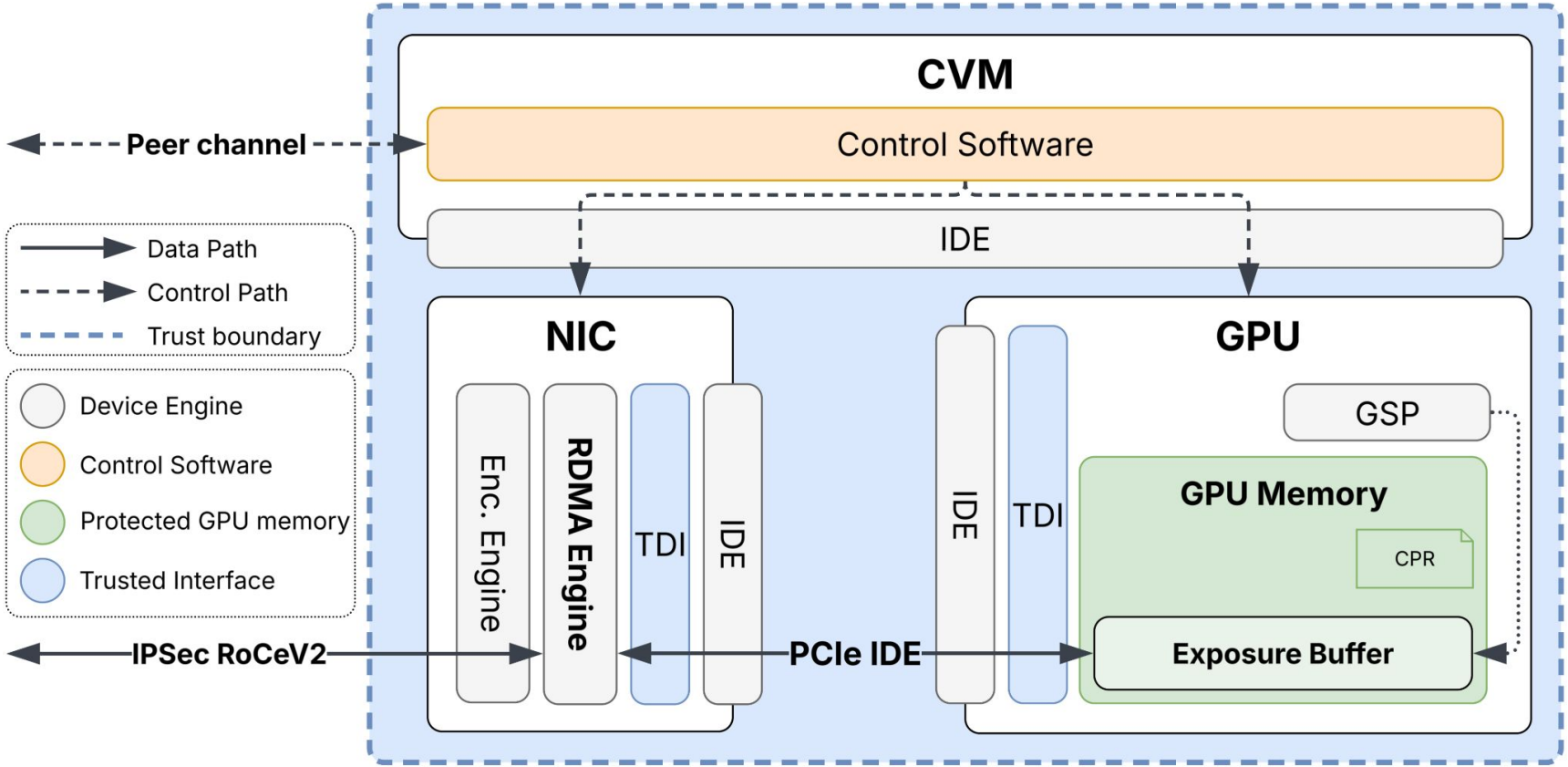
System overview



Design 1: In-GPU Bounce



Design 2: Trusted NIC



Outline

- ~~Motivation~~
- ~~Design~~
- Evaluation

Evaluation

1. How do both designs compare with the baselines in flow completion time? (Q1)
2. How do these effects translate to projected distributed-training communication time? (Q2)

Evaluation

1. Testbed:

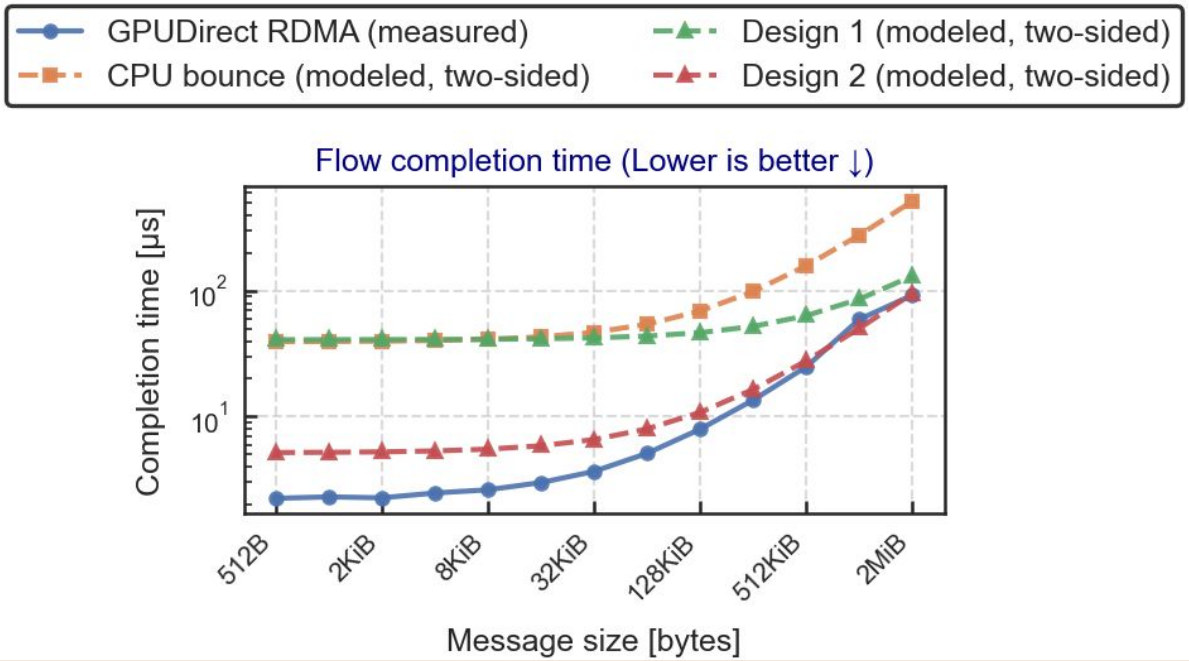
- AMD SEV-SNP confidential VM with an NVIDIA H100 GPU
- Nvidia ConnectX-7 NICs and 200 GbE Ethernet (25 GB/s nominal)
- only one GPU endpoint, complete two sided paths are projected

2. Baselines

- Plain VM GPUDirect RDMA
- Secure CPU bounce fallback in a CVM

Q1: Flow completion time

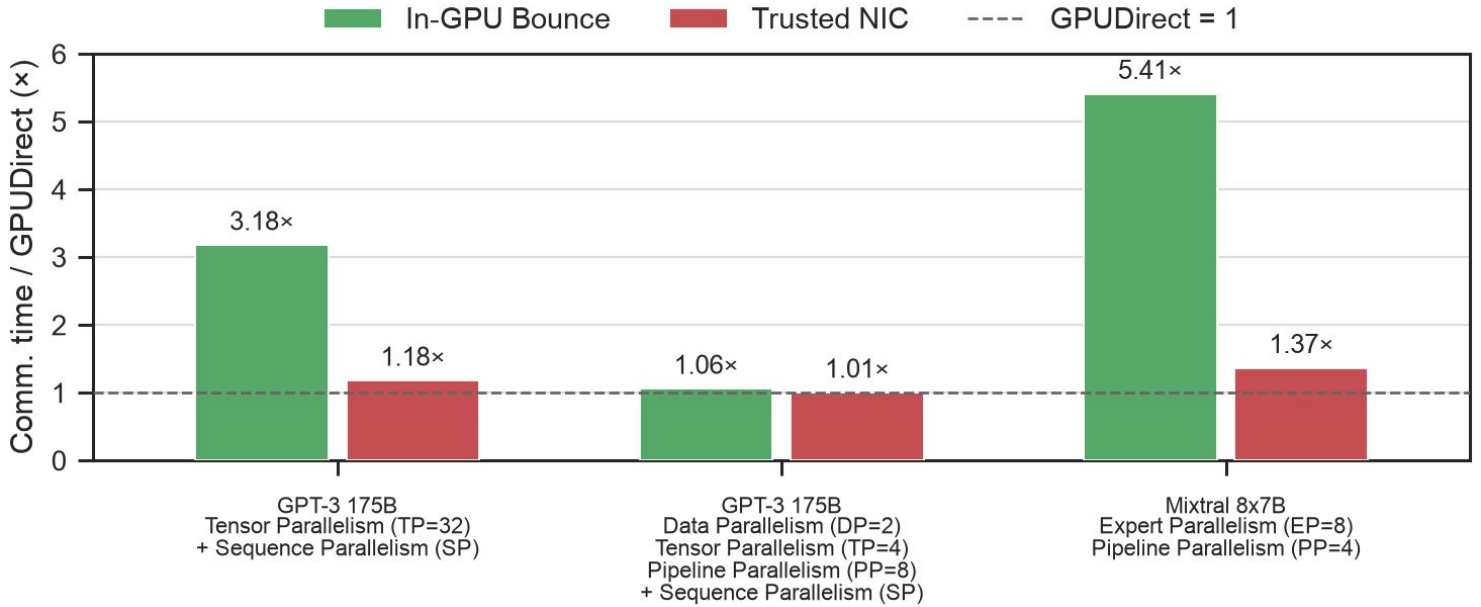
GPU Direct is measured; protected paths are projected using measurements and literature-based estimates.



The CPU bounce is throughput limited. In-GPU Bounce retains copy submission latency and Trusted NIC stays closest to GPUDirect

Q2: Distributed ML workloads

STAGE generated Chakra traces are simulated with ASTRA-sim. Physical topology: Fully connected, Logical topology: Ring.



Fixed transfer costs matter more for some communication patterns. Trusted NIC stays closer to GPUDirect bv avoiding intermediate GPU copies.

Summary

The secure CPU fallback is costly:

- staging through host memory adds copies
- CPU cryptography limits throughput
- confidential GPU command submission adds fixed latency

IronLink:

- proposes two designs that eliminate CPU payload staging and software cryptography
- **4.3 - 5.2** times faster projected communication than CPU bounce
- **2 - 23%** projected median overhead over GPUDirect

Backup

Cost Model

$$T(n) = \alpha + \frac{n}{\beta}$$

where

- T(n) is the modeled latency in seconds for transferring one payload of size n
- alpha is the fixed per-message latency independent of size
- beta is the asymptotic bandwidth

If a path consists of individual stages the following applies if those stages can be **pipelined**:

- the fixed latencies add
- the min bandwidth of all stages is used

Cost Model - GPUDirect RDMA

$$T_{\mathrm{GDR}} = 1.90\mu s + \frac{n}{23.15\mathrm{GB/s}}$$

where

- alpha was obtained using perftest ib_write_lat (write-with immediate) → latency at 32B
- beta was obtained using ib_write_bw (unidirectional, one QP) → peak average converted to GB/s

Cost Model - CPU Bounce

Three independently measured stages are composed into one model:

D2H + CPU encrypt	System-memory RDMA	CPU decrypt + H2D
$\alpha = 18.27\mu s$	$\alpha = 1.22\mu s$	$\alpha = 18.85\mu s$
$\beta = 4.87\text{GB/s}$	$\beta = 23.12\text{GB/s}$	$\beta = 4.53\text{GB/s}$

$$T_{\text{bb}}^{(1)}(n) = 19.49\mu s + \frac{n}{4.87\text{GB/s}}$$

$$T_{\text{bb}}^{(2)}(n) = 38.24\mu s + \frac{n}{4.53\text{GB/s}}$$

Cost Model - Design 1: In-GPU Bounce

We use measured primitives to parametrize the cost model

Component	Measurement	Values
GPUDirect transport	measured perftest	$\alpha = 1.90\mu s \quad \beta = 23.15\text{GB/s}$
Seal / unseal latency	confidential intra GPU D2D copy (from protected to protected vidmem)	$\alpha = 19.52\mu s \times 2$
Seal / unseal bandwidth	CPR to system memory copy. Assumption: this is a lower bound on the real CPR to non CPR copy.	$\beta = 38.78\text{GB/s}$

$$T_{\text{D1}}^{(1)}(n) = 21.42\mu s + \frac{n}{23.15\text{GB/s}} \qquad T_{\text{D1}}^{(2)}(n) = 40.94\mu s + \frac{n}{23.15\text{GB/s}}$$

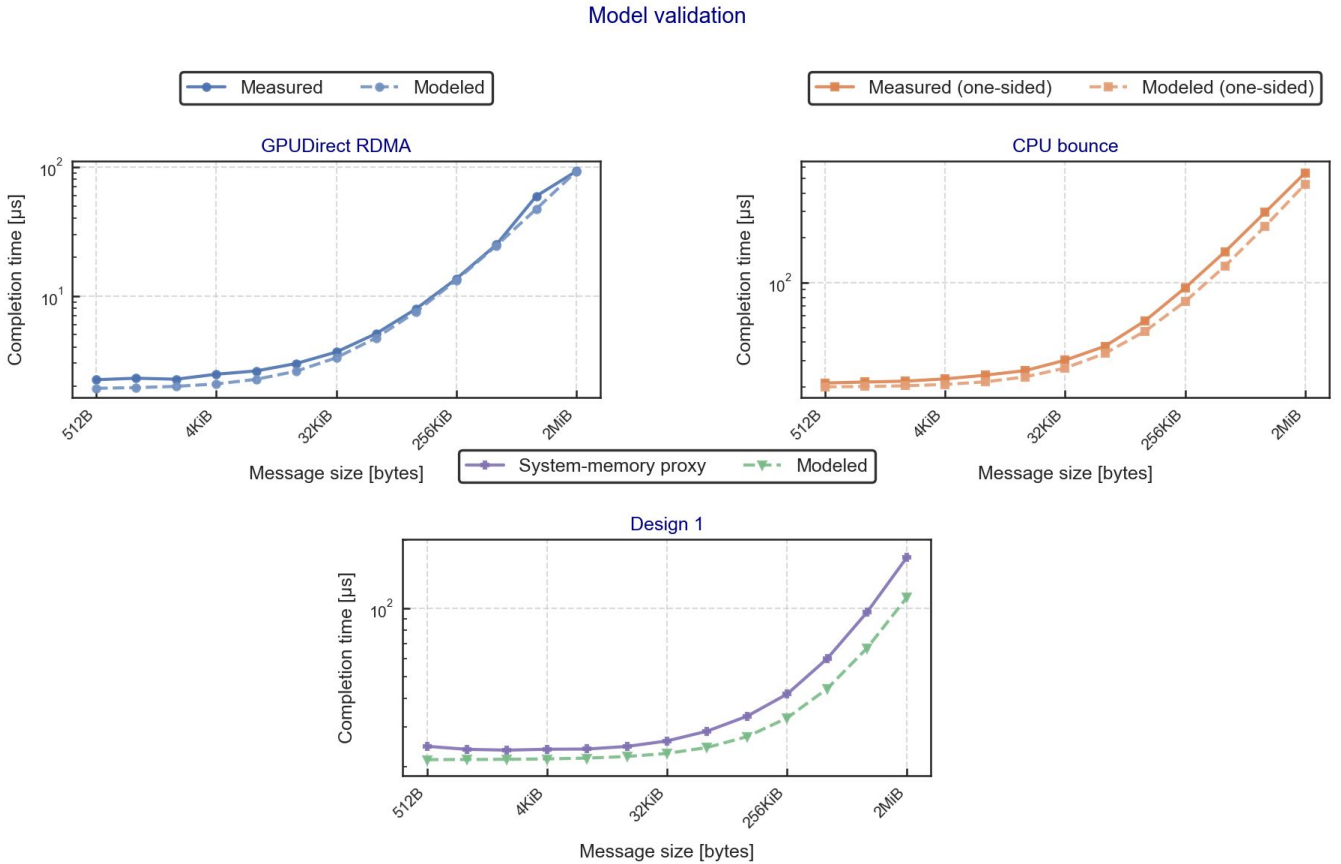
Cost Model - Design 2: Trusted NIC

We use published measurements to estimate the overhead of additional cryptography

Component	Measurement	Values
GPUDirect baseline	measured perftest	$\alpha = 1.90\mu s \quad \beta = 23.15\text{GB/s}$
IDE overhead	we assume an encryption an inline encryption engine with similar performance to published work (210 ns) we estimate upper bound for bandwidth by byte-counting	$\alpha = 6\text{PCIe ops} \times 210ns = 1.26\mu s$ $\beta = 53.80\text{GB/s}$
ESP overhead	Latency: sRDMA measured overhead Bandwidth: QRoNS fully offloaded ESP/UDP measurement	$\alpha = 2\mu s \quad \beta = 27.85\text{GB/s}$

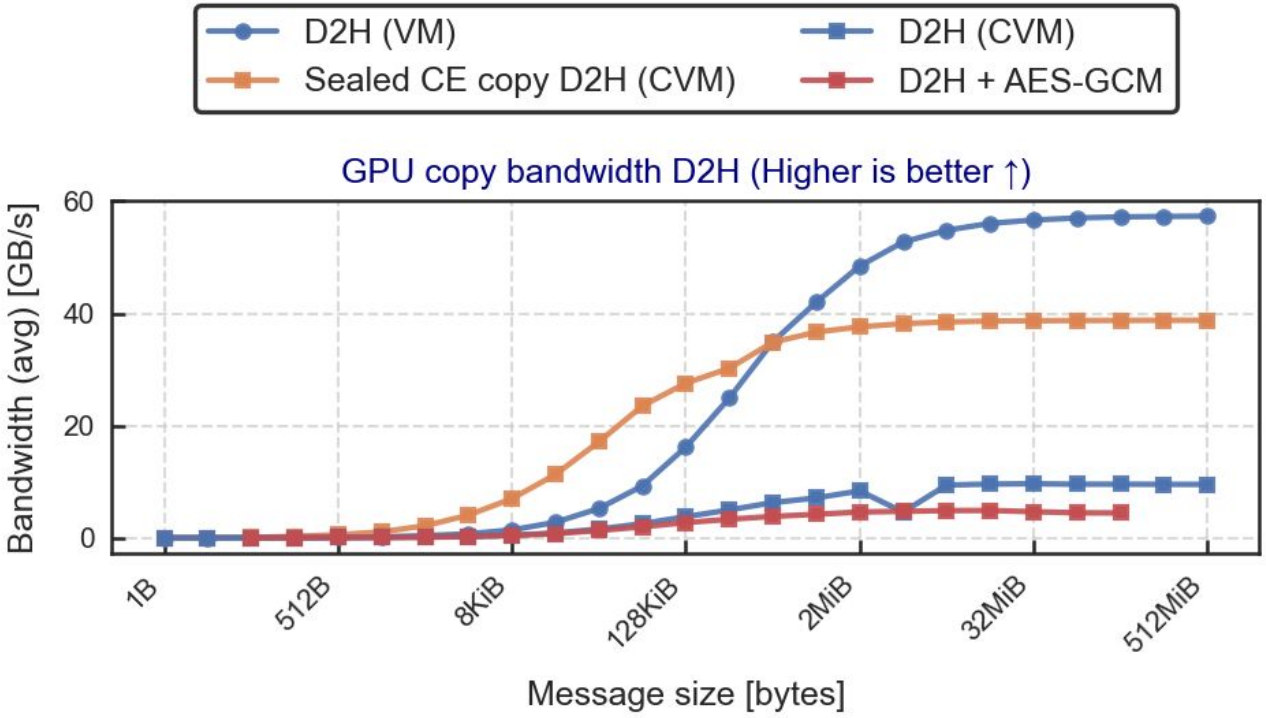
$$T_{\text{D2}}^{(2)} = 1.90+1.26+2\mu s+\frac{n}{\min(23.15, 53.80, 27.85)\text{GB/s}} = 5.16\mu s+\frac{n}{23.15\text{GB/s}}$$

Cost Model validation

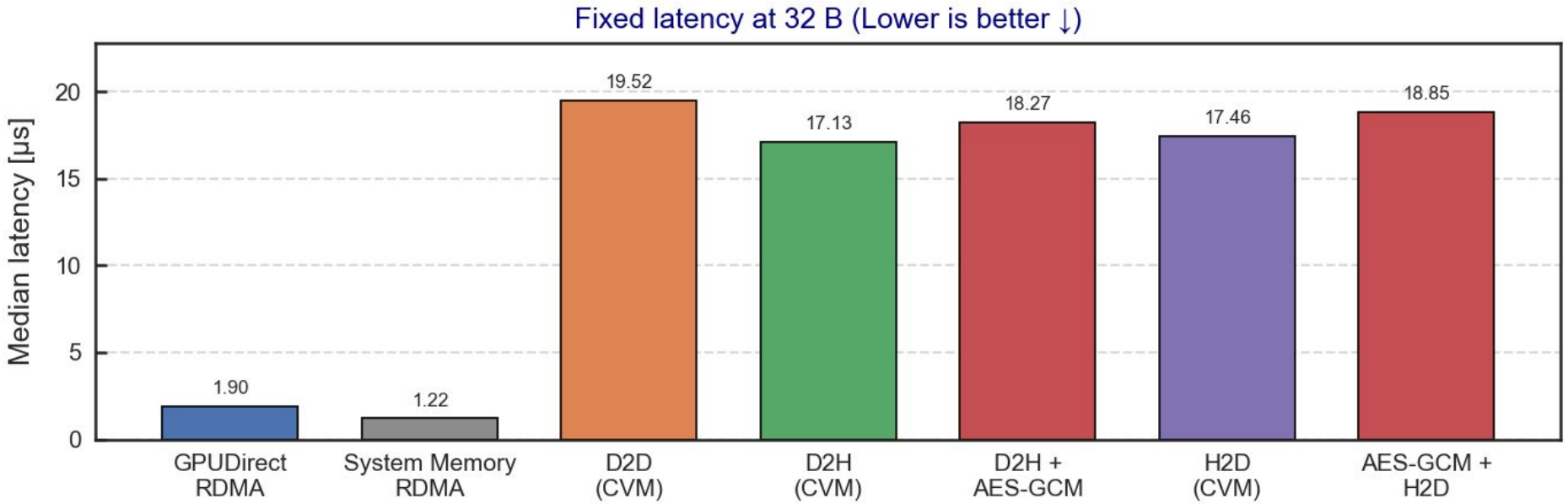


We compare GPU Direct RDMA measurements against its instantiated model, CPU bounce one sided-model against the one sided path and Design 1 against a sealed bounce from GPU memory through unprot. sysmem, followed by RDMA into the other node's unprot. sysmem.

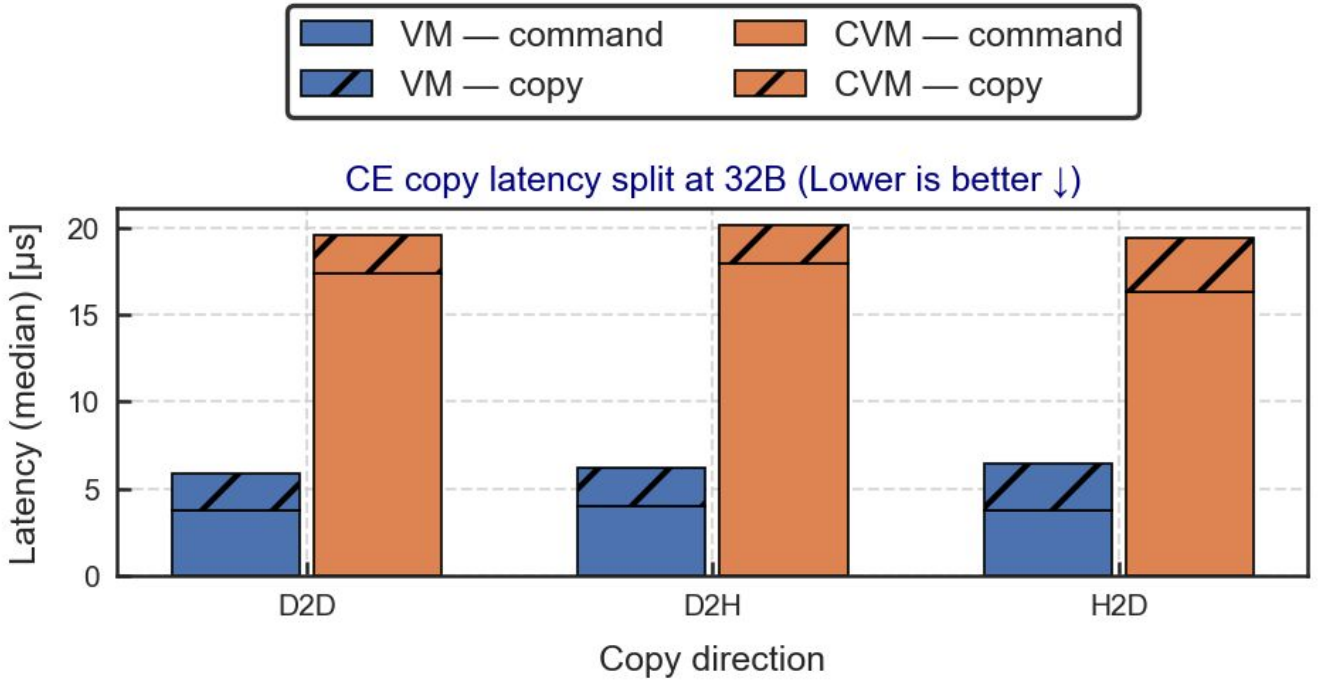
Bandwidth measurements



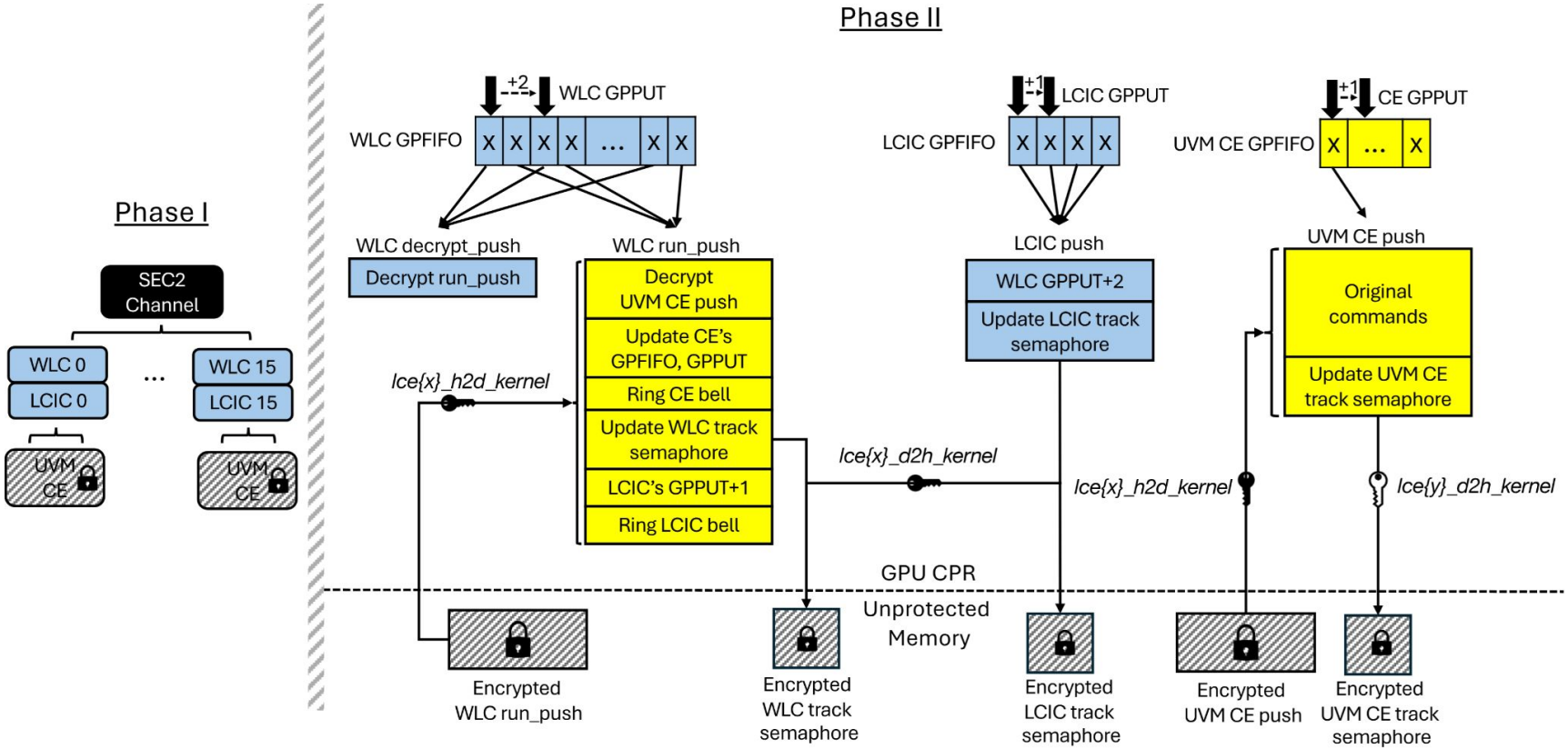
Latency measurements



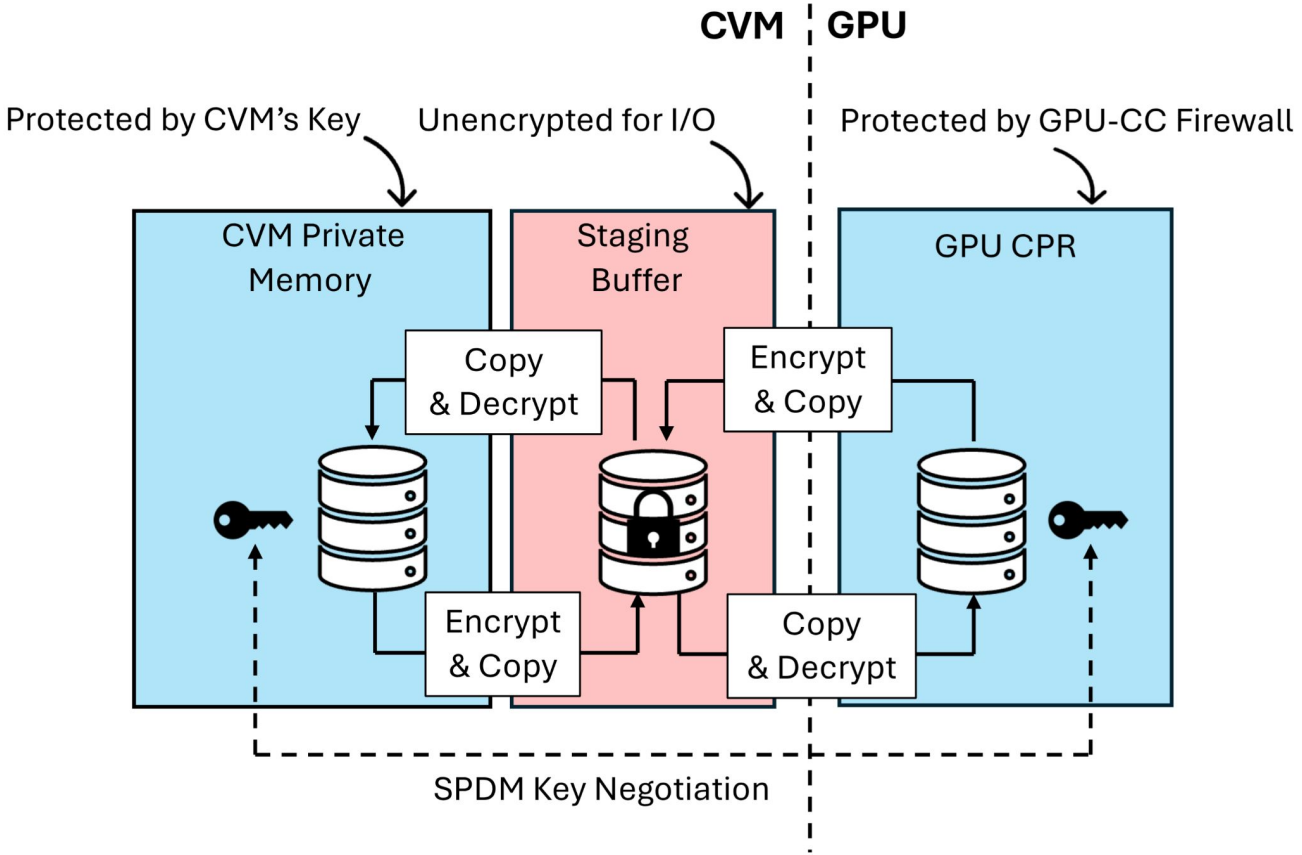
Copy Latency split



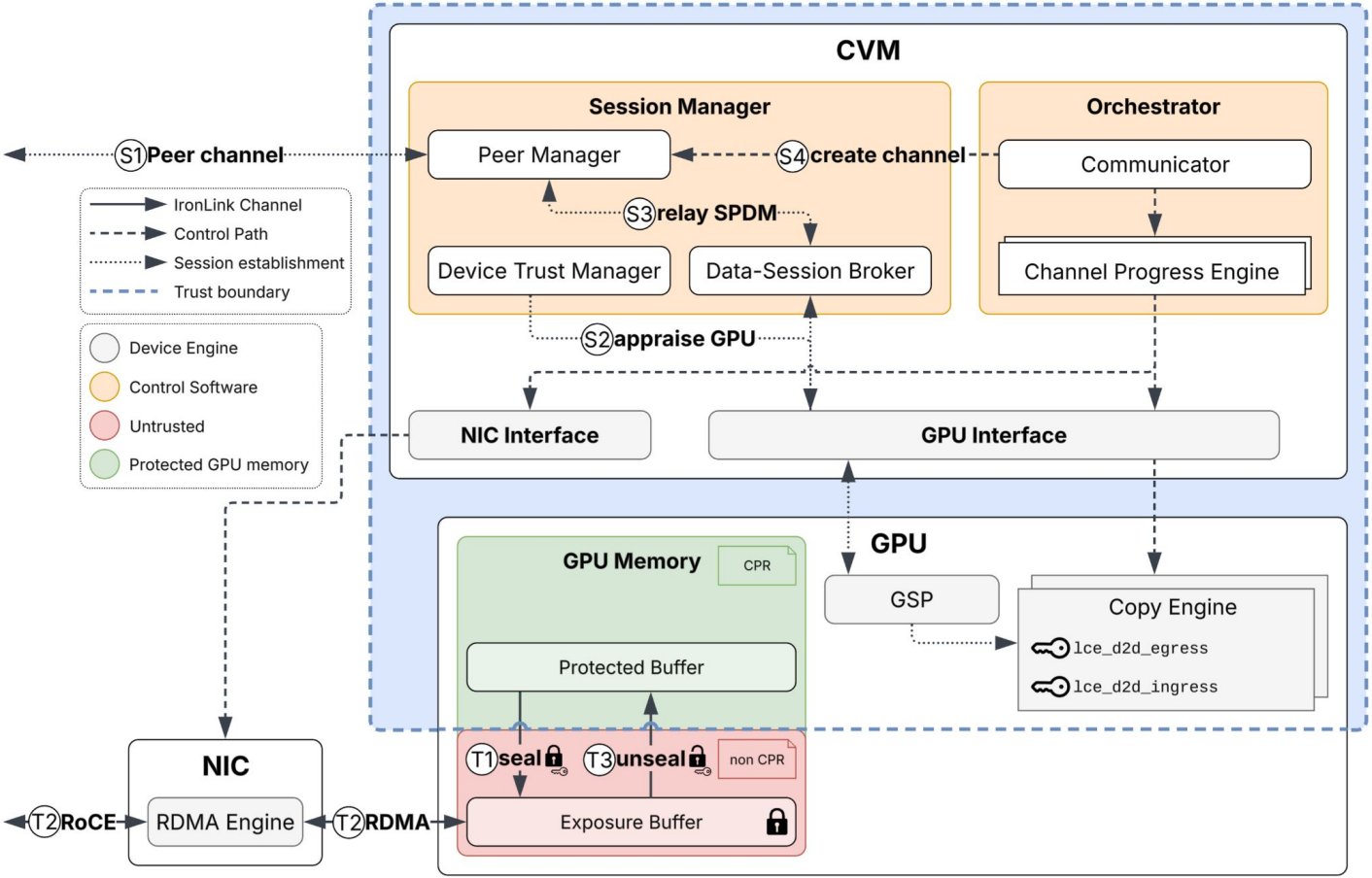
Confidential command submission



Data movement between CVM and CGPU



Detailed Design 1



Detailed Design 2

